

Security Fundamentals · Section 2

The CIA Triad and Beyond

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. Why We Need Clear Goals 1

2. Confidentiality 1

3. Integrity 2

4. Availability 3

5. Authentication, Authorization, and Accounting (AAA) 3

6. Non-Repudiation 4

7. The Trade-Offs Between Them 4

8. Applying the Goals 5

Common Mistakes 5

Summary 6

Check Your Understanding 6

Practice 7

What You Will Learn

- *Why security needs clear, named goals.*
- *The three core goals: confidentiality, integrity, and availability.*
- *Three more goals that complete the picture: authentication, authorization, and accounting (AAA).*
- *What non-repudiation means, and why it matters for money and contracts.*
- *Why the goals pull against each other, and how to balance them.*

Before You Start

Read lesson 1 first. It introduced the "three questions": can the wrong people read, change, or stop something. This lesson gives those questions their proper names and adds the goals that sit around them. No programming or networking is needed.

---

1. Why We Need Clear Goals

"Make it secure" is not a plan. Secure against what? Readable by whom? Working how often? Without clear goals, two people can both say a system is "secure" and mean completely different things, and neither can check whether it is true.

Security people solve this with a small set of named goals. When you can say which goal you are protecting, you can design for it, test it, and tell when it fails. The oldest and most useful set is three goals, known together as the ***CIA triad****: confidentiality, integrity, and availability. (The name has nothing to do with any intelligence agency; it is just the first letters.)*

Most security work, in the end, is protecting one of these three. The rest of this lesson takes them one at a time, then adds the goals that support them.

2. Confidentiality

Confidentiality means only the right people can read the information.

It is what most people first think of as "security": keeping secrets secret. Your messages, your medical records, a company's customer list, a password, all should be readable only by those allowed to see them.

*Confidentiality is broken when information reaches someone who should not have it. That is called a **data breach** or a **leak**. It can happen many ways: a stolen laptop, a database left open on the internet, an employee emailing a file to the wrong person, or an attacker reading traffic on an open Wi-Fi network.*

The main tools for confidentiality are:

- ***Encryption:** scrambling data so that only someone with the key can read it. Lesson 9 of this course, and the whole Cryptography course, go deep on this.*
- ***Access control:** deciding who may open what (lesson 7).*
- ***Authentication:** proving who someone is before letting them in (topic 5).*

A quick test for confidentiality: if this fell into a stranger's hands, would it matter? If yes, it needs protecting.

3. Integrity

Integrity means the information is correct, and is changed only by the right people, in the right way.

Confidentiality is about reading; integrity is about changing. A bank balance that anyone could edit would be useless, even if nobody could read it. Integrity makes sure that what you store and send is what you get back, unchanged by accident or by an attacker.

*Integrity is broken when data is altered in a way it should not be: an attacker changing the amount of a payment, a virus corrupting files, or a simple transmission error flipping a bit. Note that integrity covers both **malicious** change (an attacker) and **accidental** change (a bug, a faulty disk).*

The main tools for integrity are:

- ***Hashing:** a short "fingerprint" of data; if the data changes, the fingerprint changes, so you can detect tampering (Cryptography course).*
- ***Digital signatures:** proof that data came from a particular person and was not changed on the way.*
- ***Access control and validation:** only allowing changes through checks that enforce the rules.*

A quick test for integrity: if someone changed this without permission, who would be harmed, and would we even notice?

4. Availability

Availability means the system works when the people who need it need it.

Information that is perfectly secret and perfectly correct is still useless if you cannot reach it. A hospital record you cannot open during an emergency has failed, even though nobody read it or changed it.

*Availability is broken by anything that stops legitimate use: a ransomware attack that locks files, a flood of fake traffic that overwhelms a website (a **denial-of-service** attack, covered in lesson 5), a power cut, a failed hard drive, or a cut network cable. Availability is the goal most often harmed by accidents and nature, not only attackers.*

The main tools for availability are:

- ***Backups:** copies you can restore from after loss or ransomware.*
- ***Redundancy:** spare parts, servers, and power, so one failure does not stop everything.*
- ***Capacity and filtering:** enough room to handle load, and defenses against floods of fake traffic.*

A quick test for availability: if this were unreachable for an hour, a day, a week, what would it cost?

5. Authentication, Authorization, and Accounting (AAA)

*The CIA triad says what we protect. Three more goals, often called **AAA**, say how we control who does what. They support all three parts of the triad.*

- ***Authentication** answers "who are you?" It is proving identity, usually with a password, a code, a fingerprint, or a passkey. Lesson 6 is devoted to it. Weak authentication breaks confidentiality (the wrong person reads data) and integrity (the wrong person changes it).*
- ***Authorization** answers "what are you allowed to do?" Once the system knows who you are, authorization decides which actions and data you may reach. A bank teller and a bank manager sign in the same way but may do different things. Lesson 7 covers this.*
- ***Accounting** (also called **auditing**) answers "what did you do?" It is the record, the **logs**, of who did what and when. Accounting does not stop an attack by itself, but it lets you detect one, investigate it, and prove what happened. The whole defensive side of security (SOC, incident response, forensics) depends on good logs.*

*A useful way to remember the order: you **authenticate** to prove who you are, the system **authorizes** what you can do, and it **accounts** for what you did.*

6. Non-Repudiation

Non-repudiation means someone cannot credibly deny that they did something.

*If you sign a contract, you should not be able to later claim you never did. In the digital world, non-repudiation is provided mainly by **digital signatures** and solid accounting (logs). Together they tie an action to a person strongly enough that the person cannot plausibly deny it, and an outsider can be convinced.*

This matters most for money, contracts, and legal records. When a customer authorizes a payment, the bank needs proof strong enough to stand up later, not just "our system says so". Non-repudiation builds on integrity (the record was not changed) and authentication (we know who acted).

7. The Trade-Offs Between Them

The goals pull against each other. Improving one often costs another. A good security design is a balance, chosen on purpose, not an attempt to maximize every goal at once.

Push this up...	***...and this can suffer***
Confidentiality *(lock everything down, extra passwords)*	***Availability*** *and ease of use: people are slowed or locked out*
Availability *(make everything easy to reach)*	***Confidentiality****: more ways in means more ways to leak*
Integrity *(check and approve every change)*	***Availability*** *and speed: more steps, more delay*
Security *in general*	***Usability*** *and* ***cost****: the classic trade-off*

Example: a bank could require five passwords and a manager's approval for every transaction. That is very strong on confidentiality and integrity, and terrible on availability. Nobody would use it. So the bank chooses a balance: strong sign-in, extra checks for large or unusual transactions, and smooth handling of ordinary ones.

*The right balance depends on what you are protecting, which is why lesson 1 said security is about **risk**. You spend protection where the asset is valuable and the threat is real, and you keep things usable everywhere else.*

8. Applying the Goals

The goals are a checklist for thinking about any system. Take an online shop:

Goal	A question to ask	A control that helps
Confidentiality	Who can read customers' addresses and card details?	Encryption, access control
Integrity	Can anyone change a price or an order total?	Server-side checks, signatures
Availability	What happens on the busiest sales day, or under attack?	Capacity, backups, anti-flood defense
Authentication	How do customers and staff prove who they are?	Passwords plus a second factor
Authorization	Can a customer reach another customer's order?	Role and ownership checks
Accounting	If money went missing, could we trace it?	Logs of every change
Non-repudiation	Can a customer deny a purchase they made?	Signed, logged transactions

You will not design perfect answers yet, that is what the rest of the program is for, but asking these seven questions about any system already puts you ahead of most people building them. In the next lesson, we turn to how we measure and manage the dangers these goals face: threats, vulnerabilities, and risk.

Common Mistakes

1. Treating "security" as one thing. *Saying a system is "secure" means nothing until you say which goal: readable by whom, changeable by whom, available how often.*

2. Forgetting availability. *Many people equate security with secrecy and forget that a system which does not work has also failed. Backups and redundancy are security.*

3. Thinking integrity is only about attackers. *A faulty disk or a bug corrupts data too. Integrity controls catch accidental change as well as malicious change.*

4. Confusing authentication and authorization. *Proving who you are is not the same as being allowed to do something. You can be correctly identified and still not permitted.*

5. Ignoring accounting until too late. *Without logs, you cannot tell what an attacker did, or prove what a user did. Set up logging before you need it.*

6. Trying to maximize every goal. *Pushing one goal to the extreme wrecks the others and the system's usability. Choose a balance on purpose.*

Summary

Term	Remember
CIA triad	Confidentiality, integrity, availability: the three core goals
Confidentiality	Only the right people can **read** it
Integrity	Information is correct and **changed** only in the right way
Availability	The system **works** when people need it
Authentication	Proving **who you are**
Authorization	What you are **allowed to do**
Accounting	The **record** of what was done (logs)
Non-repudiation	You **cannot deny** you did it
Trade-offs	The goals pull against each other; choose a balance based on risk

Check Your Understanding

1. What do the letters in "CIA triad" stand for?
2. A virus silently corrupts a file. Which goal has been broken?
3. A website is knocked offline by a flood of fake traffic. Which goal has been broken?
4. What is the difference between authentication and authorization?
5. What does accounting (auditing) give you that the other goals do not?
6. Give one example of non-repudiation mattering in real life.
7. Why can't a system simply maximize all the goals at once? Give one trade-off.

Answers

1. Confidentiality, Integrity, and Availability.
2. Integrity: the data was changed in a way it should not have been (even without an attacker reading it).
3. Availability: legitimate users cannot reach the site when they need it.
4. Authentication proves who you are; authorization decides what you are allowed to do once you are identified.

5. A record of who did what and when (logs), so an attack or a user's actions can be detected, investigated, and proven after the fact.
6. Any sensible example: signing a contract and not being able to deny it; authorizing a bank payment that the bank can later prove you made.
7. Because the goals pull against each other. For example, pushing confidentiality up with many passwords and approvals reduces availability and usability, so nobody can get their work done.

Practice

Pick one system you know well: a messaging app, an online bank, your email, or a school portal.

1. Write one sentence for each of the three CIA goals, saying what it means for that system (for example, "Confidentiality: only I should read my messages").
2. For the same system, say how you **authenticate** (prove who you are), and name one thing you are **not authorized** to do.
3. Name one place where that system trades a goal away for usability (for example, staying signed in so you don't type a password every time, which trades some confidentiality for availability).

A Worked Example

For an email account:

1. Confidentiality: only I should read my emails. Integrity: nobody should be able to alter an email I received or send one as me. Availability: I should be able to reach my email when I need it.
2. I authenticate with a password and a code sent to my phone. I am not authorized to read another person's mailbox on the same mail service.
3. "Keep me signed in" on my own phone trades a little confidentiality (anyone holding my unlocked phone can read my mail) for availability and convenience (I don't re-enter my password each time).

Course mark icon from Material Symbols by Google, Apache License 2.0. NIST, CIS, ISO, OWASP, and MITRE ATT&CK are the trademarks of their respective owners, used here only to refer to their published work.